

第8章 网络通信

建议学时:4-6 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言

本章学习目标

- 掌握 Java 的 TCP 编程:ServerSocket 与 Socket,与 C 的 socket 系列函数对照
- 掌握多客户端处理模型:每连接一线程的经典架构
- 掌握 UDP 编程:DatagramSocket 的发送与接收
- 掌握 HttpClient:GET/POST、超时、错误处理(JDK11+)
- 理解网络编程的坑:阻塞、半关闭、超时、粘包与拆包
- 树立网络安全红线:不信任远端输入、防注入

从 C 到 Java:网络编程的手工活变少了

C 的网络编程是"仪式感"的代名词:socket() → bind() → listen() → accept(),每步都要检查返回值、处理 errno,再手写 read/write 循环。Java 把这套流程压缩成两个类:服务端 ServerSocket(accept 即完成 listen+accept),客户端 Socket(connect 完成握手)。连接的读写直接就是第7章的 InputStream/OutputStream——网络流和文件流是同一套 API,这是 Java 的设计哲学:一切都是流。

最大的思维差异是阻塞。C 的 recv 阻塞,Java 的 in.read 同样阻塞——没有数据就一直挂着。C 程序员用 select/poll/epoll 解决多连接;Java 经典方案是"每连接一个线程":accept 到连接就 new 一个线程去读,主循环继续 accept。这背后是第9章要展开的线程模型。现代高并发(Netty、虚拟线程)是另一套,但"一连接一线程"依然是理解一切的起点。

HTTP 是 C 程序员的"手工活重灾区":手写请求行、解析响应头、处理 chunked——Java 的 java.net.http.HttpClient (JDK11+)把 GET/POST、超时、重定向、异步全部内置,十几行完成 C 里上百行的活。生产调第三方 REST API,这是标准姿势。

安全是网络章节的必修课:C 时代的心智是"自己的程序互相通信";今天的网络程序面对的是不可信输入——JSON 注入、路径穿越、无认证端口裸奔。本章每个示例都带安全笔记,红线在第8.7节。

学习顺序:TCP 先建立连接模型(\$8.2-8.3),UDP 快速对照(\$8.4),然后 HTTP(\$8.5),最后是坑与安全(\$8.6-8.7)。

前置知识自查

- C 网络基础:socket/bind/listen/accept/connect、recv/send、TCP 三次握手概念
- 第7章:流式读写与 try-with-resources(网络读写复用同一套)
- 第4章:方法;基本的多线程直觉(第9章将完整展开)

本章学习路线

1. **第一阶段(约 1.5 小时):**\$8.2-8.3 跑通"回显服务器 + 客户端",用两个终端观察收发
2. **第二阶段(约 1 小时):**\$8.4 UDP 对照;\$8.5 HttpClient 请求真实接口
3. **第三阶段(约 1.5 小时):**\$8.6-8.7 坑与安全,完成章末实验
4. **验收标准:**能写出 TCP 回显服务器;能用 HttpClient 带超时地 GET 并解析响应;能说出三条安全红线

本章知识导图

- §8.1 网络编程总览:与 C socket 家族对照
- §8.2 TCP 编程:ServerSocket / Socket
- §8.3 多客户端:每连接一线程
- §8.4 UDP 编程:DatagramSocket
- §8.5 HttpClient:GET / POST / 超时
- §8.6 网络编程的坑:阻塞 / 半关闭 / 粘包
- §8.7 安全红线:不可信输入

§8.1 网络编程总览

8.1.1 与 C 的 socket 家族对照

C 函数	Java 对应	说明
socket(AF_INET, SOCK_STREAM, 0)	new ServerSocket(port) / new Socket(host, port)	构造即创建,类型由类决定
bind + listen	ServerSocket 构造器内部完成	一次搞定
accept	server.accept() → Socket	阻塞等待连接
connect	new Socket(host, port)	构造即连接(可带超时)
recv / send	socket.getInputStream() / getOutputStream()	就是第7章的流!
close	socket.close()	关闭连接
errno	IOException(异常体系)	异常替代错误码

★ 重点:网络连接 = 两端各有一对流

TCP 连接建立后,两端各拿到"对方的输入流/输出流":你写出去的数据出现在对端的读流里,对端写的数据从你的读流进来。所以网络编程 = 第7章的流编程 + 连接的建立与关闭管理。别被"网络"两个字吓住,它只是流的另一种数据源。

§8.2 TCP 编程:回显服务器

示例 8-1 TCP 回显服务器(单连接版)

```

import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.OutputStreamWriter;
import java.net.ServerSocket;
import java.net.Socket;
import java.nio.charset.StandardCharsets;

public class EchoServer {
    public static void main(String[] args) throws IOException {
        int port = 9000;
        // 构造即 bind + listen;C 的三步压缩成一步
        try (ServerSocket server = new ServerSocket(port)) {
            System.out.println("监听端口 " + port + " ...");

            // accept 阻塞:等一个客户端连上来
            try (Socket client = server.accept()) {
                System.out.println("客户端接入: " + client.getRemoteSocketAddress());

                // 网络流:显式 UTF-8,与文件读写同一套姿势
                var in = new BufferedReader(new InputStreamReader(
                    client.getInputStream(), StandardCharsets.UTF_8));
                var out = new BufferedWriter(new OutputStreamWriter(
                    client.getOutputStream(), StandardCharsets.UTF_8));

                String line;
                while ((line = in.readLine()) != null) {
                    System.out.println("收到: " + line);
                    out.write("回显: " + line + "\n"); // 原样弹回
                    out.flush(); // 缓冲流必须刷
                }
                System.out.println("客户端断开");
            }
        }
    }
}

```

■ 示例 8-2 TCP 客户端

```

import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.OutputStreamWriter;
import java.net.Socket;
import java.nio.charset.StandardCharsets;

public class EchoClient {
    public static void main(String[] args) throws IOException {
        // 构造即 connect;SocketTimeoutException 表示连不上
        try (Socket sock = new Socket("127.0.0.1", 9000)) {
            var in = new BufferedReader(new InputStreamReader(
                sock.getInputStream(), StandardCharsets.UTF_8));
            var out = new BufferedWriter(new OutputStreamWriter(
                sock.getOutputStream(), StandardCharsets.UTF_8));

            out.write("你好,服务器\n");
            out.flush();
            System.out.println("服务器回应: " + in.readLine()); // 阻塞等回应
        }
    }
}
// 运行顺序:先启动 EchoServer,再启动 EchoClient

```

⚠ 易错点 1:阻塞与超时

readLine 没有数据时 **永远阻塞**——它不会超时。生产代码必须设置超时: `sock.setSoTimeout(3000)`, 超时后抛 `SocketTimeoutException`(`IOException` 子类), 循环里 catch 后决定重试或断开。另一个隐蔽点:客户端不主动 close, 服务器端 readLine 就一直等, 看不到"客户端断开"——用 finally 或 try-with-resources 保证 close。

§8.3 多客户端:每连接一线程

8.3.1 架构:accept 循环 + 工作线程

单连接版只能服务一个客户端。经典多客户端模型: **主循环只做 accept; 每接一个连接, 交给一个新线程处理, 主循环立刻回去等下一个**。这正是 C 里 pthread 版服务器的 Java 对应, 线程创建用 Executors(第9章详解, 这里先用最简单的)。

示例 8-3 多客户端聊天服务器(核心结构)

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.net.ServerSocket;
import java.net.Socket;
import java.nio.charset.StandardCharsets;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class ChatServer {
    // 线程池:固定 10 个工作线程,避免无限制创建线程(第9章详解)
    private static final ExecutorService pool = Executors.newFixedThreadPool(10);

    public static void main(String[] args) throws IOException {
        try (ServerSocket server = new ServerSocket(9100)) {
            System.out.println("聊天服务器启动, 端口 9100");
            while (true) {
                // 主循环只做一件事:等连接
                Socket client = server.accept();
                // 把连接交给工作线程,立刻回去 accept 下一个
                pool.submit(() -> handle(client));
            }
        }
    }

    private static void handle(Socket client) {
        // 每个客户端一个线程:阻塞读自己的流,互不干扰
        try (client; // try-with-resources 关连接
            var in = new BufferedReader(new InputStreamReader(
                client.getInputStream(), StandardCharsets.UTF_8))) {
            System.out.println "[" + Thread.currentThread().getName()
                + "] 新用户: " + client.getRemoteSocketAddress());
            String line;
            while ((line = in.readLine()) != null) {
                System.out.println "[" + Thread.currentThread().getName()
                    + "] 广播: " + line);
                // 完整版:把消息转发给所有在线的客户端(需要维护连接列表)
            }
            System.out.println("用户离开: " + client.getRemoteSocketAddress());
        } catch (IOException e) {
            System.out.println("连接异常: " + e.getMessage());
        }
    }
}
```

工程建议:线程模型的演进路线

"一连接一线程"的正确打开方式是线程池,不是裸 new Thread :裸创建在高并发下会把内存与 CPU 打爆。演进路线:一连接一线程(好理解)→ 线程池(控制资源)→ NIO 事件驱动(Netty,大量连接少量活跃)→ 虚拟线程(JDK21+,极轻量线程,回到"一连接一线程"的写法但无成本)。理解第一代,是理解后面的前提。

§8.4 UDP 编程

8.4.1 无连接的数据报

UDP 不需要 accept/connect 握手: DatagramPacket 装着"数据 + 目标地址",DatagramSocket 负责发送与接收。数据报有大小上限(约 64KB)、可能丢失或乱序。适合:DNS、音视频、游戏位置同步、心跳探测。

示例 8-4 UDP 收发

```
import java.net.DatagramPacket;
import java.net.DatagramSocket;
import java.net.InetAddress;
import java.nio.charset.StandardCharsets;

public class UdpDemo {
    public static void main(String[] args) throws Exception {
        // 发送端:一次发一包
        try (DatagramSocket sender = new DatagramSocket()) {
            byte[] data = "ping".getBytes(StandardCharsets.UTF_8);
            DatagramPacket pkt = new DatagramPacket(data, data.length,
                InetAddress.getByAddress("127.0.0.1"), 9200);
            sender.send(pkt);
            System.out.println("已发送 UDP 包");
        }

        // 接收端:绑定端口,等待数据报(先启动接收端)
        try (DatagramSocket receiver = new DatagramSocket(9200)) {
            byte[] buf = new byte[1024];
            DatagramPacket pkt = new DatagramPacket(buf, buf.length);
            receiver.receive(pkt); // 阻塞等包
            String msg = new String(pkt.getData(), 0, pkt.getLength(),
                StandardCharsets.UTF_8);
            System.out.println("收到: " + msg + " 来自 "
                + pkt.getAddress() + ":" + pkt.getPort());
        }
    }
}
```

易错点 2:UDP 的四个"不保证"

UDP 不保证:送达、顺序、不重复、不丢包。业务需要可靠性就必须在应用层补(序号、重传、确认),或者干脆用 TCP。经典错误:把 UDP 当 TCP 用,丢包后一脸茫然。选型判断:丢一包无所谓的心跳→ UDP;一字节都不能丢的(转账)→ TCP。另外 UDP 包默认有大小上限,应用层要控制单包数据量。

§8.5 HttpClient:现代 HTTP 客户端

8.5.1 GET 与 POST

java.net.http.HttpClient (JDK11+)是内置的 HTTP 客户端: HttpRequest 描述请求,HttpClient 发送,HttpResponse 承载响应。GET 简单、POST 带 body;超时与重定向策略在客户端构建时配置。C 里手写 HTTP 的苦活在这里是标准库能力。

示例 8-5 HttpClient 实战

```

import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.time.Duration;

public class HttpDemo {
    public static void main(String[] args) throws Exception {
        // 客户端:全局一个,配置超时与跟随重定向
        HttpClient client = HttpClient.newBuilder()
            .connectTimeout(Duration.ofSeconds(5)) // 连接超时
            .followRedirects(HttpClient.Redirect.NORMAL)
            .build();

        // ---- GET ----
        HttpRequest getReq = HttpRequest.newBuilder()
            .uri(URI.create("https://httpbin.org/get"))
            .timeout(Duration.ofSeconds(10)) // 请求超时
            .GET()
            .build();
        HttpResponse<String> resp =
            client.send(getReq, HttpResponse.BodyHandlers.ofString());
        System.out.println("GET 状态码: " + resp.statusCode()); // 200
        System.out.println("GET 响应体前 120 字符: "
            + resp.body().substring(0, 120));

        // ---- POST(JSON body)----
        String json = "{\"name\":\"张三\",\"score\":88}";
        HttpRequest postReq = HttpRequest.newBuilder()
            .uri(URI.create("https://httpbin.org/post"))
            .header("Content-Type", "application/json")
            .POST(HttpRequest.BodyPublishers.ofString(json))
            .build();
        HttpResponse<String> postResp =
            client.send(postReq, HttpResponse.BodyHandlers.ofString());
        System.out.println("POST 状态码: " + postResp.statusCode());
        System.out.println("POST 响应体前 120 字符: "
            + postResp.body().substring(0, 120));
    }
}

```

★ 重点:状态码与错误处理

send 只在“网络层失败”(连不上、超时)抛 IOException;业务失败(404、500)是正常的 HTTP 响应。必须检查 statusCode():2xx 成功、3xx 重定向(可自动跟随)、4xx 客户端错、5xx 服务端错。生产姿势:封装“响应校验 + 重试策略 + 熔断”,但初学先记住:不检查状态码 = 把错误当成功。

§8.6 网络编程的坑

坑	现象	对策
阻塞	read 永远等不到数据,程序挂死	setSoTimeout 超时 + catch SocketTimeoutException
半关闭	一端不再写,另一端 readLine 返回 null 才知道;close 直接断双方	用 shutdownOutput() 只关写方向;约定 报文结束标记
粘包/拆包	一次 read 读到多条消息(粘包)或一条 被拆成多次(拆包)	应用层协议必须定边界:按行(文本)/ 长度 前缀 / JSON 对象
半包写	write 一次没写完,对方 read 到半截数 据	write 的返回值检查;BufferedWriter 全行 写 + flush
端口占用	BindException: Address already in use	换端口;服务端优雅关闭 (ServerSocket.close 释放)

示例 8-6 粘包演示与按行协议修复

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.net.Socket;
import java.nio.charset.StandardCharsets;

public class StickyDemo {
    // 场景:客户端一口气 write 三条消息,服务端一次 read 可能全读到
    // 修复:应用层约定"一行一条消息",用 readLine 按行切分
    public static void main(String[] args) throws IOException {
        // 假设已拿到连接 socket
        Socket sock = new Socket("127.0.0.1", 9100);
        sock.setSoTimeout(3000); // 3 秒没数据就超时
        try (var in = new BufferedReader(new InputStreamReader(
            sock.getInputStream(), StandardCharsets.UTF_8))) {
            String line;
            while ((line = in.readLine()) != null) {
                System.out.println("一条消息: " + line);
            }
        } catch (java.net.SocketTimeoutException e) {
            System.out.println("读超时:本次无更多数据(约定:超时即结束)");
        }
    }
}
```

⚠ 易错点 3:协议设计是网络程序的第一设计决策

TCP 是字节流,没有"消息"概念——消息边界必须由应用层协议定义。三条主路:① 文本行协议(每行一条,readLine);② 长度前缀(4 字节长度 + payload);③ 自描述格式(JSON/Protobuf,生产首选)。C 程序员常用"固定结构体 + sizeof"——Java 里同样可行(长度前缀),但二进制协议的字节序要显式处理(DataOutputStream 的 writeInt 是网络序,别手搓)。

§8.7 安全红线

⚠ 易错点 4:网络安全的四条红线

1. 不信任任何远端输入:长度检查、类型检查、白名单校验;网络数据当"可能被攻击者精心构造"对待
2. 防止注入:把网络数据拼进 SQL/命令/路径前必须转义或参数化;路径穿越(../)用 normalize 后检查前缀
3. 认证与授权:暴露端口的服务必须有认证(令牌/API Key),别裸奔;生产环境监听 0.0.0.0 前想清楚
4. 加密传输:涉及敏感数据的传输用 HTTPS/TLS(Java 的 `HttpsURLConnection`/`HttpClient` 原生支持),不要自己实现加密

示例 8-7 红线落地:输入校验模板

```
import java.util.regex.Pattern;

public class SanitizeDemo {
    // 学号白名单:只允许 7 位数字;其余一律拒绝
    private static final Pattern STUDENT_ID = Pattern.compile("\\d{7}");

    // 校验模板:长度 + 内容双重把关,失败抛异常而不是返回"猜的值"
    static String sanitizeNick(String raw) {
        if (raw == null || raw.length() > 20 || raw.length() < 1) {
            throw new IllegalArgumentException("昵称长度必须在 1-20");
        }
        String trimmed = raw.trim();
        if (trimmed.contains("
") || trimmed.contains("
")) {
            throw new IllegalArgumentException("昵称不能包含换行");    // 防协议注入
        }
        return trimmed;
    }

    static boolean isStudentId(String s) {
        return s != null && STUDENT_ID.matcher(s).matches();
    }
}
```

本章速查表

主题	要点
TCP 服务端	<code>new ServerSocket(port) → accept() → 读写流 → close</code> ;三步合一
TCP 客户端	<code>new Socket(host, port)</code> 即连接; <code>getInputStream/getOutputStream</code> 就是流
阻塞	<code>read</code> 无数据永远等; <code>setSoTimeout(ms) + catch SocketTimeoutException</code>
多客户端	<code>accept</code> 循环 + 线程池提交连接处理;别裸 <code>new Thread</code>
UDP	<code>DatagramPacket(数据+地址) + DatagramSocket</code> ;无连接;不保证送达/顺序/不丢
UDP 选型	心跳/音视频/丢包无所谓 → UDP;一字节不能丢 → TCP
HttpClient	<code>HttpRequest + HttpClient.send + HttpResponse</code> ;body 用 <code>BodyHandlers.ofString()</code>
状态码	404/500 是正常响应不是异常;必须检查 <code>statusCode</code> ;超时才抛 <code>IOException</code>
粘包/拆包	TCP 无消息边界;协议三选一:行协议 / 长度前缀 / JSON
半关闭	<code>close</code> 断双向;只停写用 <code>shutdownOutput</code> ;对方 EOF = <code>readLine</code> null
超时配置	HttpClient: <code>connectTimeout</code> + 请求 <code>timeout</code> 都要设
安全红线	不信任输入 / 防注入 / 要认证 / 加密传输(HTTPS)

最小必会清单

- 能默写 TCP 回显服务器骨架(ServerSocket/accept/流/关闭)
- 能写出带超时的客户端读取循环
- 能说出"每连接一线程 + 线程池"架构并画出主循环流程
- 能说出 UDP 与 TCP 的选型判断
- 能用 HttpClient 完成带超时的 GET 并检查状态码
- 能解释粘包成因并给出三种协议对策
- 能说出四条网络安全红线

自测题

Q 1. 服务器 accept 到一个连接后,客户端发来 100 字节,服务端一次 read(buf) 可能读到多少?为什么?

✓ 参考答案

可能是 1~100 之间的任意值(还可能拆成多次读):TCP 是字节流,数据到达的时间与分割不受发送方 write 次数约束。所以必须循环 read 直到读够协议要求的量(长度前缀协议)或读到行结束符(行协议),不能假设一次 read 就是一条消息。

Q 2. 客户端突然断电,服务端的 readLine 会怎样?怎么及时发现?

✓ 参考答案

断电时 TCP 无正常 FIN,服务端 readLine 一直阻塞,可能永远等不到。对策:setSoTimeout 让读超时;应用层心跳(定期发 ping,超过 N 次没回应判死)。OS 层面 TCP keepalive 是另一个兜底,但间隔太长,生产一般靠应用层心跳。

Q 3. HttpClient.send 抛异常与状态码 500 有什么区别?

✓ 参考答案

send 抛 IOException:网络层失败(连不上、DNS 解析失败、超时)——程序不知道服务器是否处理了请求。状态码 500:请求到达服务器,服务器内部出错——服务器知道、也返回了明确答复。处理策略不同:前者可重试(幂等请求),后者看业务(500 往往重试也无用)。

Q 4. 服务器需要同时服务 5000 个在线连接,但多数连接长期空闲。为什么"裸线程每连接"不合适?现代解法?

✓ 参考答案

每线程约 1MB 栈(第5章),5000 线程占几 GB 虚拟内存,且线程切换开销大。传统解法:Netty 的 NIO 事件驱动(一个线程管大量连接);JDK21+ 虚拟线程:轻量到百万级,恢复"一连接一线程"的简单写法。生产判断:连接量级千级以下 → 线程池够用;万级以上 → Netty 或虚拟线程。

Q 5. 客户端发来字符串,服务器要把它拼进 SQL 查询。有什么风险?怎么办?

✓ 参考答案

SQL 注入:字符串里带 ' OR 1=1 -- 之类的内容会改写查询语义。对策:使用参数化查询(PreparedStatement 的 ? 占位),绝不字符串拼接 SQL;同时做输入白名单校验(如学号必须是数字)。这是第8.7节红线的具体落地。

Q 6. UDP 协议为什么不适合传输"订单支付确认"?

✓ 参考答案

UDP 不保证送达与顺序:支付确认丢了,用户以为没支付成功,业务就错了。UDP 适合"丢失可容忍、实时性优先"的场景(游戏位置、语音、心跳)。业务关键数据用 TCP(或基于 UDP 的应用层可靠性协议 QUIC,但那仍是"可靠传输"的语义)。

章末实验题:命令聊天室 + 天气查询

实验 8:命令聊天室与 HTTP 查询

实验目标:编写两部分程序:① 多客户端命令行聊天室(服务端 + 客户端);② 用 HttpClient 查询公共 API 并解析结果。综合运用本章全部知识点。

实验要求:

- 任务 1:服务端维护在线客户端集合,收到消息后广播给所有在线者(覆盖 §8.2-8.3 TCP 与多线程)
- 任务 2:每连接一线程用线程池;客户端断开时从集合移除(覆盖 §8.3)
- 任务 3:客户端提供昵称,发送格式 "昵称: 消息",服务端广播时带发送者(覆盖 §8.2 客户端)
- 任务 4:客户端设置 5 秒读超时,超时后提示并继续(覆盖 §8.6 超时处理)
- 任务 5:服务端与客户端之间采用"一行一条消息"的行协议(覆盖 §8.6 粘包对策)
- 任务 6:用 HttpClient GET 查询 <https://httpbin.org/ip>,解析出 origin 字段打印(覆盖 §8.5 HTTP)
- 任务 7:POST 一条 JSON 到 <https://httpbin.org/post>,打印回显的 json 字段(覆盖 §8.5 POST)
- 任务 8:所有网络数据按"不可信输入"处理:行长度限制、异常捕获、优雅关闭(覆盖 §8.7 安全)

实验环境:JDK 17+;需要联网(任务 6-7);两个终端分别启动服务端与客户端。

第一步 跑通单客户端回显(示例 8-1/8-2),再加在线集合与广播

第二步 播时对"发送者"本人要不要回显,自己决定并在注释里说明理由

第三步 务 6-7 先 curl 试通接口,再写 Java 代码对照

参考答案要点

```

import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.OutputStreamWriter;
import java.net.ServerSocket;
import java.net.Socket;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.nio.charset.StandardCharsets;
import java.time.Duration;
import java.util.Map;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

/** 命令聊天室 + HTTP 查询 — 第8章综合实验参考答案
 * 覆盖:TCP 多客户端、线程池、广播、超时、行协议、HttpClient、安全红线
 */
public class ChatRoom {

    // 在线集合:并发安全,value 是写给该客户端的流
    private static final Map<String, BufferedWriter> clients =
        new ConcurrentHashMap<>();
    private static final ExecutorService pool = Executors.newFixedThreadPool(10);

    public static void main(String[] args) throws Exception {
        if (args.length > 0 && args[0].equals("server")) {
            startServer(9200);
        } else if (args.length > 0 && args[0].equals("client")) {
            startClient("127.0.0.1", 9200, "游客" + (int) (Math.random() * 1000));
        } else if (args.length > 0 && args[0].equals("http")) {
            httpTasks();
        } else {
            System.out.println("用法: java ChatRoom server | client | http");
        }
    }

    // ---- 服务端 ----
    private static void startServer(int port) throws IOException {
        try (ServerSocket server = new ServerSocket(port)) {
            System.out.println("聊天室监听 " + port);
            while (true) {
                Socket client = server.accept();    // 主循环只 accept
                pool.submit(() -> handleClient(client));
            }
        }
    }

    private static void handleClient(Socket client) {
        String nick = null;
        try (client;
            var in = new BufferedReader(new InputStreamReader(
                client.getInputStream(), StandardCharsets.UTF_8));
            var out = new BufferedWriter(new OutputStreamWriter(
                client.getOutputStream(), StandardCharsets.UTF_8))) {
            // 第一行是昵称(行协议:一条消息 = 一行)
            nick = in.readLine();
            if (nick == null || nick.length() > 20) {
                return;    // 红线:限制输入长度
            }
            clients.put(nick, out);
            broadcast("系统", nick + " 加入聊天室");
            String line;
            while ((line = in.readLine()) != null) {

```

```

        if (line.length() > 200) {
            line = line.substring(0, 200);    // 红线:消息长度上限
        }
        broadcast(nick, line);
    }
} catch (IOException e) {
    System.out.println("连接异常: " + e.getMessage());
} finally {
    if (nick != null) {
        clients.remove(nick);
        broadcast("系统", nick + " 离开");
    }
}
}

// 广播:遍历在线集合,逐个写(写失败就移除该用户)
private static void broadcast(String from, String msg) {
    String packet = from + ": " + msg + "\n";    // 行协议
    for (var entry : clients.entrySet()) {
        try {
            entry.getValue().write(packet);
            entry.getValue().flush();
        } catch (IOException e) {
            clients.remove(entry.getKey());    // 写不动的用户踢出
        }
    }
}

// ---- 客户端 ----
private static void startClient(String host, int port, String nick)
    throws IOException {
    try (Socket sock = new Socket(host, port)) {
        sock.setSoTimeout(5000);    // 读超时 5 秒
        var in = new BufferedReader(new InputStreamReader(
            sock.getInputStream(), StandardCharsets.UTF_8));
        var out = new BufferedWriter(new OutputStreamWriter(
            sock.getOutputStream(), StandardCharsets.UTF_8));
        out.write(nick + "\n");    // 先报到昵称
        out.flush();

        // 读线程:收广播(超时只是提示,不断线)
        Thread reader = new Thread(() -> {
            try {
                String line;
                while ((line = in.readLine()) != null) {
                    System.out.println(line);
                }
            } catch (java.net.SocketTimeoutException e) {
                System.out.println("(5 秒无消息,继续等待)");
            } catch (IOException e) {
                System.out.println("连接已断开");
            }
        });
        reader.setDaemon(true);
        reader.start();

        // 主线程:读键盘发送
        try (var kb = new BufferedReader(
            new InputStreamReader(System.in, StandardCharsets.UTF_8))) {
            String line;
            while ((line = kb.readLine()) != null) {
                out.write(line + "\n");
                out.flush();
            }
        }
    }
}

// ---- HTTP 任务 ----

```

```

private static void httpTasks() throws Exception {
    HttpClient client = HttpClient.newBuilder()
        .connectTimeout(Duration.ofSeconds(5))
        .build();

    // 任务 6:GET 拿本机出口 IP
    HttpResponse<String> ipResp = client.send(
        HttpRequest.newBuilder()
            .uri(URI.create("https://httpbin.org/ip"))
            .timeout(Duration.ofSeconds(10))
            .GET().build(),
        HttpResponse.BodyHandlers.ofString());
    System.out.println("GET /ip 状态: " + ipResp.statusCode());
    // 简单解析:{ "origin": "x.x.x.x" } 取冒号后到引号前的部分
    String body = ipResp.body();
    String origin = body.substring(body.indexOf("origin") + 10,
        body.lastIndexOf(''));
    System.out.println("本机出口 IP: " + origin);

    // 任务 7:POST JSON
    HttpResponse<String> postResp = client.send(
        HttpRequest.newBuilder()
            .uri(URI.create("https://httpbin.org/post"))
            .header("Content-Type", "application/json")
            .POST(HttpRequest.BodyPublishers.ofString(
                "{\"chat\":\"你好\"}"))
            .build(),
        HttpResponse.BodyHandlers.ofString());
    System.out.println("POST 状态: " + postResp.statusCode());
    System.out.println("POST 回显: " + postResp.body()
        .substring(0, Math.min(200, postResp.body().length())));
}
}

```

设计说明:在线集合用 ConcurrentHashMap(第9章的并发容器)——多个线程同时读写在线列表,普通 HashMap 会崩;广播对“写失败”的连接做移除,避免僵尸用户阻塞线程池;客户端读线程用 daemon 线程,主线程退出时不阻塞。任务 6 的 JSON 解析是“手搓”的(用 indexOf),生产必须用 Jackson 等库(第13章),这里故意演示“能读但别写”。扩展方向:私聊指令、历史消息、心跳保活、HTTPS + 令牌认证。

下一章预告:第9章 并发与异常。线程的生命周期、同步与锁、异常体系与最佳实践——网络程序的并发问题,和 Java 的异常世界观,都在这一章。